MET CS 688

Two Clustering Use Cases

Leila Ghaedi – Fall 2024

Creator: cemagraphics | Credit: Getty Images/iStockphoto

Customer Segmentation

- In this project, we will conduct unsupervised clustering on customer records from a grocery company's database.
- Customer segmentation involves grouping customers based on similar behavior/attributes.
- The goal is to categorize customers into segments, optimizing their significance to the business.
- This segmentation allows for tailored product modifications based on unique customer needs and behaviors, facilitating the business in addressing the concerns of different customer types.

https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis

Customer Segmentation
Import Libraries

In [1]:

```
#Customer Segmentation Project
import mglearn
import numpy as np
import pandas as pd
import datetime
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib
import matplotlib.pyplot as plt
from matplotlib import colors
import seaborn as sns
from yellowbrick.cluster import KElbowVisualizer
from sklearn.cluster import KMeans
from mpl_toolkits.mplot3d import Axes3D
from sklearn.cluster import AgglomerativeClustering
from matplotlib.colors import ListedColormap
from sklearn import metrics
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.simplefilter(action='ignore', category=DeprecationWarning)
warnings.simplefilter(action='ignore', category=RuntimeWarning)
```

Customer Segmentation
Load the dataset

In [2]:
```
# Load the data and do the exploratory data analysis
customer_data = pd.read_csv("marketing_campaign.csv", sep="\t")
print("Number of records:", len(customer_data))
customer_data.head()
```

```
Number of records: 2240
```

Out[2]:

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	NumWebVisitsMonth	Ac
0	5524	1957	Graduation	Single	58138.0	0	0	04-09-2012	58	635	...	7	
1	2174	1954	Graduation	Single	46344.0	1	1	08-03-2014	38	11	...	5	
2	4141	1965	Graduation	Together	71613.0	0	0	21-08-2013	26	426	...	4	
3	6182	1984	Graduation	Together	26646.0	1	0	10-02-2014	26	11	...	6	
4	5324	1981	PhD	Married	58293.0	1	0	19-01-2014	94	173	...	5	

5 rows × 29 columns

Customer Segmentation

```
In [3]:  1  customer_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   2240 non-null   int64
 1   Year_Birth           2240 non-null   int64
 2   Education            2240 non-null   object
 3   Marital_Status       2240 non-null   object
 4   Income               2216 non-null   float64
 5   Kidhome              2240 non-null   int64
 6   Teenhome             2240 non-null   int64
 7   Dt_Customer          2240 non-null   object
 8   Recency              2240 non-null   int64
 9   MntWines             2240 non-null   int64
 10  MntFruits            2240 non-null   int64
 11  MntMeatProducts      2240 non-null   int64
 12  MntFishProducts      2240 non-null   int64
 13  MntSweetProducts     2240 non-null   int64
 14  MntGoldProds         2240 non-null   int64
 15  NumDealsPurchases    2240 non-null   int64
 16  NumWebPurchases      2240 non-null   int64
 17  NumCatalogPurchases  2240 non-null   int64
 18  NumStorePurchases    2240 non-null   int64
 19  NumWebVisitsMonth    2240 non-null   int64
 20  AcceptedCmp3         2240 non-null   int64
 21  AcceptedCmp4         2240 non-null   int64
 22  AcceptedCmp5         2240 non-null   int64
 23  AcceptedCmp1         2240 non-null   int64
 24  AcceptedCmp2         2240 non-null   int64
 25  Complain             2240 non-null   int64
 26  Z_CostContact        2240 non-null   int64
 27  Z_Revenue            2240 non-null   int64
 28  Response             2240 non-null   int64
dtypes: float64(1), int64(25), object(3)
memory usage: 507.6+ KB
```

Customer Segmentation

In [4]:
```
1 customer_data.describe().T
```

Out[4]:

	count	mean	std	min	25%	50%	75%	max
ID	2240.0	5592.159821	3246.662198	0.0	2828.25	5458.5	8427.75	11191.0
Year_Birth	2240.0	1968.805804	11.984069	1893.0	1959.00	1970.0	1977.00	1996.0
Income	2216.0	52247.251354	25173.076661	1730.0	35303.00	51381.5	68522.00	666666.0
Kidhome	2240.0	0.444196	0.538398	0.0	0.00	0.0	1.00	2.0
Teenhome	2240.0	0.506250	0.544538	0.0	0.00	0.0	1.00	2.0
Recency	2240.0	49.109375	28.962453	0.0	24.00	49.0	74.00	99.0
MntWines	2240.0	303.935714	336.597393	0.0	23.75	173.5	504.25	1493.0
MntFruits	2240.0	26.302232	39.773434	0.0	1.00	8.0	33.00	199.0
MntMeatProducts	2240.0	166.950000	225.715373	0.0	16.00	67.0	232.00	1725.0
MntFishProducts	2240.0	37.525446	54.628979	0.0	3.00	12.0	50.00	259.0
MntSweetProducts	2240.0	27.062946	41.280498	0.0	1.00	8.0	33.00	263.0
MntGoldProds	2240.0	44.021875	52.167439	0.0	9.00	24.0	56.00	362.0
NumDealsPurchases	2240.0	2.325000	1.932238	0.0	1.00	2.0	3.00	15.0
NumWebPurchases	2240.0	4.084821	2.778714	0.0	2.00	4.0	6.00	27.0
NumCatalogPurchases	2240.0	2.662054	2.923101	0.0	0.00	2.0	4.00	28.0
NumStorePurchases	2240.0	5.790179	3.250958	0.0	3.00	5.0	8.00	13.0
NumWebVisitsMonth	2240.0	5.316518	2.426645	0.0	3.00	6.0	7.00	20.0
AcceptedCmp3	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.00	1.0
AcceptedCmp4	2240.0	0.074554	0.262728	0.0	0.00	0.0	0.00	1.0
AcceptedCmp5	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.00	1.0
AcceptedCmp1	2240.0	0.064286	0.245316	0.0	0.00	0.0	0.00	1.0
AcceptedCmp2	2240.0	0.013393	0.114976	0.0	0.00	0.0	0.00	1.0
Complain	2240.0	0.009375	0.096391	0.0	0.00	0.0	0.00	1.0
Z_CostContact	2240.0	3.000000	0.000000	3.0	3.00	3.0	3.00	3.0
Z_Revenue	2240.0	11.000000	0.000000	11.0	11.00	11.0	11.00	11.0
Response	2240.0	0.149107	0.356274	0.0	0.00	0.0	0.00	1.0

Customer Segmentation Data

- The data consists of 2240 records, where each record has 29 attributes.
- Attributes:
  1. Personal Information (ID, Year of Birth, Education, Marital Status,…)
  2. Product (MntWines, MntFruits, MntMeatProducts,…)
  3. Promotion (NumDealsPurchases, AcceptedCmp1, AcceptedCmp2, …)
  4. Channel (NumWebPurchases, NumCatalogPurchases,…)

Customer Segmentation

Personal Information:

ID: Customer's unique identifier
Year_Birth: Customer's birth year
Education: Customer's education level
Marital_Status: Customer's marital status
Income: Customer's yearly household income
Kidhome: Number of children in customer's household
Teenhome: Number of teenagers in customer's household
Dt_Customer: Date of customer's enrollment with the company
Recency: Number of days since customer's last purchase
Complain: 1 if the customer complained in the last 2 years, 0 otherwise

Customer Segmentation

Products:

MntWines: Amount spent on wine in last 2 years

MntFruits: Amount spent on fruits in last 2 years

MntMeatProducts: Amount spent on meat in last 2 years

MntFishProducts: Amount spent on fish in last 2 years

MntSweetProducts: Amount spent on sweets in last 2 years

MntGoldProds: Amount spent on gold in last 2 years

Customer Segmentation

Promotion:

NumDealsPurchases: Number of purchases made with a discount

AcceptedCmp1: 1 if customer accepted the offer in the 1st campaign, 0 otherwise

AcceptedCmp2: 1 if customer accepted the offer in the 2nd campaign, 0 otherwise

AcceptedCmp3: 1 if customer accepted the offer in the 3rd campaign, 0 otherwise

AcceptedCmp4: 1 if customer accepted the offer in the 4th campaign, 0 otherwise

AcceptedCmp5: 1 if customer accepted the offer in the 5th campaign, 0 otherwise

Response: 1 if customer accepted the offer in the last campaign, 0 otherwise

Customer Segmentation

Channel:

- NumWebPurchases: Number of purchases made through the company's website
- NumCatalogPurchases: Number of purchases made using a catalogue
- NumStorePurchases: Number of purchases made directly in stores
- NumWebVisitsMonth: Number of visits to company's website in the last month

Customer Segmentation
Categorical Variables

In [5]:
```
# Categorical Variable levels
print("Marital_Status:\n", customer_data["Marital_Status"].value_counts(), "\n")
print("Education:\n", customer_data["Education"].value_counts())
```

```
Marital_Status:
 Marital_Status
Married     864
Together    580
Single      480
Divorced    232
Widow        77
Alone         3
Absurd        2
YOLO          2
Name: count, dtype: int64

Education:
 Education
Graduation    1127
PhD            486
Master         370
2n Cycle       203
Basic           54
Name: count, dtype: int64
```

Customer Segmentation
Missing Data

- There are 24 missing data points in Income field.
- Since the missing data is just few records and in one attribute, we can drop the missing records.

In [6]:
```
# Drop missing data points
customer_data = customer_data.dropna()
print("The total number of recrds after removing the rows with missing values:", len(customer_data))
```

```
The total number of recrds after removing the rows with missing values: 2216
```

Customer Segmentation
Formatting

- Dt_Customer that indicates the date a customer joined the database is not parsed as DateTime.
- We need to encode categorical and binary features as well.

In [7]:
```
# date time formatting
customer_data["Dt_Customer"] = pd.to_datetime(customer_data["Dt_Customer"], format='%d-%m-%Y')
```

Customer Segmentation
Feature Engineering

- Create a “Tenure” field shows the number of days the customers started to shop in the store relative to the last recorded date.
- Extract “Age” at 2012 based on Year_Birth.
- Create feature "Spent" indicating the total amount spent by the customer in all categories over the span of two years.
- Create another feature "Living_With" out of "Marital_Status" to extract the living situation of couples.
- Create a feature "Children" to indicate total children in a household that is, kids and teenagers.
- Creating feature indicating "Family_Size"
- Create a feature "Parent" to indicate parenthood status
- Create three categories in the "Education" by simplifying its value counts.
- Drop some of the redundant features.

Customer Segmentation
Feature Engineering

In [8]:
```
# Feature Engineering Steps
```

In [9]:
```
dates = []
for i in customer_data["Dt_Customer"]:
    i = i.date()
    dates.append(i)
#Dates of the newest and oldest recorded customer
print("The newest customer's enrolment date in therecords:",max(dates))
print("The oldest customer's enrolment date in the records:",min(dates))
```

```
The newest customer's enrolment date in therecords: 2014-06-29
The oldest customer's enrolment date in the records: 2012-07-30
```

In [10]:
```
# Create a feature "Tenure"
days = []
d1 = max(dates) # compare with the newest customer date
for i in dates:
    delta = d1 - i
    days.append(delta.days)
customer_data["Tenure"] = days
customer_data["Tenure"] = pd.to_numeric(customer_data["Tenure"], errors="coerce")

```

Customer Segmentation
Feature Engineering

In [11]:
```
#Create "Age" at 2012
customer_data["Age"] = 2012- customer_data["Year_Birth"]

```

In [12]:
```
#Remove outliers by setting a cap on Age and income.

customer_data = customer_data[(customer_data["Age"]<100)]
customer_data = customer_data[(customer_data["Income"]<600000)]
print("The total number of recrds after removing the routliers:", len(customer_data))
```

```
The total number of recrds after removing the routliers: 2212
```

In [13]:
```
#Total spend on all items
customer_data["Spent"] = customer_data["MntWines"]+ \
                         customer_data["MntFruits"]+ \
                         customer_data["MntMeatProducts"]+ \
                         customer_data["MntFishProducts"]+ \
                         customer_data["MntSweetProducts"]+ \
                         customer_data["MntGoldProds"]


```

Customer Segmentation
Feature Engineering

In [14]:
```
# living situation by marital status "Alone" vs "Partner"
customer_data["Living_With"]=customer_data["Marital_Status"].\
                             replace({"Married":"Partner", "Together":"Partner",\
                                      "Absurd":"Alone", "Widow":"Alone", "YOLO":"Alone",\
                                      "Divorced":"Alone", "Single":"Alone",})
```

In [15]:
```
#total children living in the household
customer_data["Children"]=customer_data["Kidhome"]+customer_data["Teenhome"]

# total members in the householde
customer_data["Family_Size"] = customer_data["Living_With"].\
                               replace({"Alone": 1, "Partner":2})+ customer_data["Children"]
# parenthood
customer_data["Parent"] = np.where(customer_data.Children> 0, 1, 0)
```

In [16]:
```
#Segmenting education levels in three groups
customer_data["Education"]=customer_data["Education"].\
                           replace({"Basic":"Undergraduate","2n Cycle":"Undergraduate", \
                                    "Graduation":"Graduate", "Master":"Postgraduate",\
                                    "PhD":"Postgraduate"})
```

Customer Segmentation
Feature Engineering

In [17]:
```
# drop  the redundant features
to_drop = ["Marital_Status",  "Year_Birth", "ID", "Z_CostContact","Z_Revenue", "Dt_Customer"]
customer_data = customer_data.drop(to_drop, axis=1)
```

Customer Segmentation-Feature Engineering

```
In [18]: 1 customer_data.describe().T
```

Out[18]:

	count	mean	std	min	25%	50%	75%	max
Income	2212.0	51958.810579	21527.278844	1730.0	35233.5	51371.0	68487.00	162397.0
Kidhome	2212.0	0.441682	0.536955	0.0	0.0	0.0	1.00	2.0
Teenhome	2212.0	0.505877	0.544253	0.0	0.0	0.0	1.00	2.0
Recency	2212.0	49.019439	28.943121	0.0	24.0	49.0	74.00	99.0
MntWines	2212.0	305.287523	337.322940	0.0	24.0	175.5	505.00	1493.0
MntFruits	2212.0	26.329566	39.744052	0.0	2.0	8.0	33.00	199.0
MntMeatProducts	2212.0	167.029837	224.254493	0.0	16.0	68.0	232.25	1725.0
MntFishProducts	2212.0	37.648734	54.772033	0.0	3.0	12.0	50.00	259.0
MntSweetProducts	2212.0	27.046564	41.090991	0.0	1.0	8.0	33.00	262.0
MntGoldProds	2212.0	43.925859	51.706981	0.0	9.0	24.5	56.00	321.0
NumDealsPurchases	2212.0	2.324593	1.924507	0.0	1.0	2.0	3.00	15.0
NumWebPurchases	2212.0	4.088156	2.742187	0.0	2.0	4.0	6.00	27.0
NumCatalogPurchases	2212.0	2.672242	2.927542	0.0	0.0	2.0	4.00	28.0
NumStorePurchases	2212.0	5.806510	3.250939	0.0	3.0	5.0	8.00	13.0
NumWebVisitsMonth	2212.0	5.321429	2.425597	0.0	3.0	6.0	7.00	20.0
AcceptedCmp3	2212.0	0.073689	0.261323	0.0	0.0	0.0	0.00	1.0
AcceptedCmp4	2212.0	0.074141	0.262060	0.0	0.0	0.0	0.00	1.0
AcceptedCmp5	2212.0	0.072785	0.259842	0.0	0.0	0.0	0.00	1.0
AcceptedCmp1	2212.0	0.064195	0.245156	0.0	0.0	0.0	0.00	1.0
AcceptedCmp2	2212.0	0.013562	0.115691	0.0	0.0	0.0	0.00	1.0
Complain	2212.0	0.009042	0.094678	0.0	0.0	0.0	0.00	1.0
Response	2212.0	0.150542	0.357683	0.0	0.0	0.0	0.00	1.0
Tenure	2212.0	353.714286	202.494886	0.0	180.0	356.0	529.00	699.0
Age	2212.0	43.086347	11.701599	16.0	35.0	42.0	53.00	72.0
Spent	2212.0	607.268083	602.513364	5.0	69.0	397.0	1048.00	2525.0
Children	2212.0	0.947559	0.749466	0.0	0.0	1.0	1.00	3.0
Family_Size	2212.0	2.593128	0.906236	1.0	2.0	3.0	3.00	5.0
Parent	2212.0	0.714286	0.451856	0.0	0.0	1.0	1.00	1.0

Customer Segmentation-Feature Engineering

In [19]:

```
# only numerical values
to_drop = ["Education",  "Living_With"]
customer_data_num= customer_data.drop(to_drop, axis=1)

#correlation matrix
corrmat= customer_data_num.corr()
plt.figure(figsize=(20,20))
sns.heatmap(corrmat,annot=True, center=0)
```

Customer Segmentation
Data Pre-Processing

- The following steps are applied to preprocess the data:
  - Label encoding the categorical features
  - Scaling the features using the standard scaler
  - Creating a subset dataframe for dimensionality reduction

Customer Segmentation-Data Pre-Processing

In [20]:
```
# Save a copy of the data for profiling before encoding categories
customer_data_copy=customer_data
# Encode all categorical features
#Label Encoding the object dtypes.
LE=LabelEncoder()
for i in ['Education', 'Living_With']:
    customer_data[i]=customer_data[[i]].apply(LE.fit_transform)

```

In [21]:
```
# Scale and save in a new data frame scaled_df
scaler = StandardScaler()
scaler.fit(customer_data)
scaled_df = pd.DataFrame(scaler.transform(customer_data),columns= customer_data.columns )
scaled_df.head()
```

Out[21]:

	Education	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts	...	A
0	-0.893586	0.287105	-0.822754	-0.929699	0.310353	0.977660	1.552041	1.690293	2.453472	1.483713	...	
1	-0.893586	-0.260882	1.040021	0.908097	-0.380813	-0.872618	-0.637461	-0.718230	-0.651004	-0.634019	...	
2	-0.893586	0.913196	-0.822754	-0.929699	-0.795514	0.357935	0.570540	-0.178542	1.339513	-0.147184	...	
3	-0.893586	-1.176114	1.040021	-0.929699	-0.795514	-0.872618	-0.561961	-0.655787	-0.504911	-0.585335	...	
4	0.571657	0.294307	1.040021	-0.929699	1.554453	-0.392257	0.419540	-0.218684	0.152508	-0.001133	...	

5 rows × 30 columns

Customer Segmentation-Data Pre-Processing PCA

- In this problem, there are 30 factors based on which the final clustering will be done.
- These factors are basically attributes or features.
- Many of these features are correlated, and hence redundant.
- This is why we will be performing dimensionality reduction on the features before feeding them to a clustering model.

Customer Segmentation-Data Pre-Processing
PCA

In [22]:

```
# PCA for dimensionality reduction
#Initiating PCA to reduce dimentions aka features to 3
pca = PCA(n_components=3)
pca.fit(scaled_df)
PCA_df = pd.DataFrame(pca.transform(scaled_df), columns=(["column1","column2", "column3" ]))
PCA_df.describe().T
```

Out[22]:

	count	mean	std	min	25%	50%	75%	max
column1	2212.0	-2.569775e-17	2.960584	-5.924200	-2.556958	-0.861992	2.264132	8.537421
column2	2212.0	4.497106e-17	1.717093	-4.159480	-1.348848	-0.159602	1.248615	6.064053
column3	2212.0	1.284887e-17	1.389386	-3.186204	-0.786302	-0.163359	0.411128	8.260273

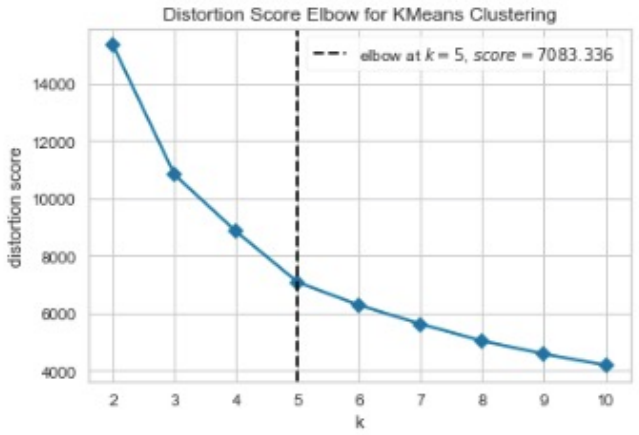
Customer Segmentation- Clustering

1. Elbow Method to determine the number of clusters to be formed
2. Clustering using Agglomerative Clustering
3. Examining the clusters formed visually

Customer Segmentation- Clustering Elbow Method

In [23]:
```
# Elbow method to find numbers of clusters (based on PCA_df)
print('Elbow Method for PCA data set:')
Elbow_M = KElbowVisualizer(KMeans(random_state=42),k=10, timings=False)
Elbow_M.fit(PCA_df)
Elbow_M.show()
```

```
Elbow Method for PCA data set:
```



Out[23]:
```
<Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>
```

Customer Segmentation- Clustering
Agglomerative Clustering

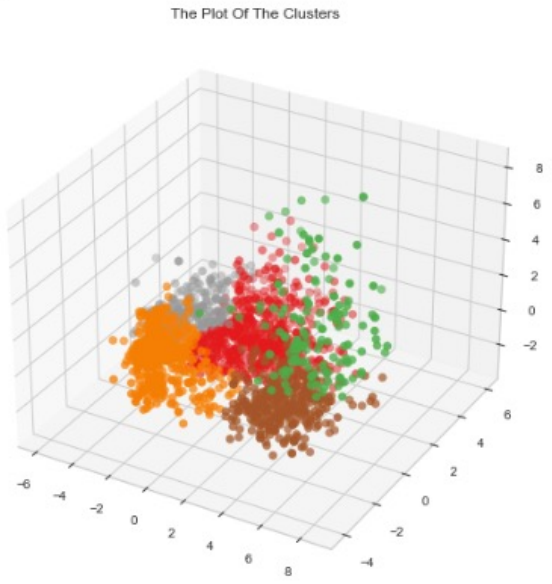
In [24]:

```
#Initiating the Agglomerative Clustering model
AC = AgglomerativeClustering(n_clusters=5)
# fit model and predict clusters
yhat_AC = AC.fit_predict(PCA_df)
PCA_df["Clusters"] = yhat_AC
#Adding the Clusters feature to the orignal dataframe.
customer_data["Clusters"]= yhat_AC
customer_data_copy["Clusters"]= yhat_AC
```

Customer Segmentation- Clustering
Agglomerative Clustering

In [25]:

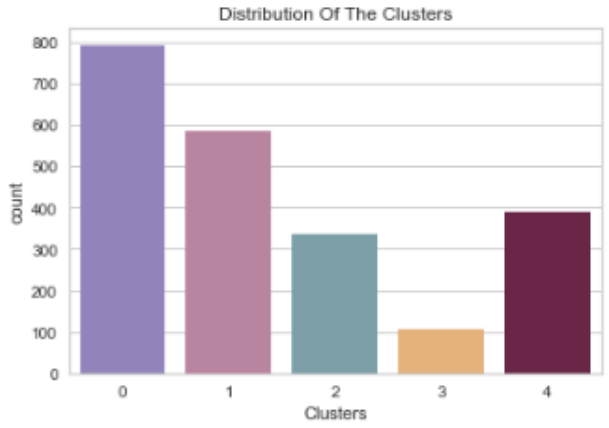
```
from matplotlib import cm
tab20_5 = cm.get_cmap("Set1", lut = 5)
x =PCA_df["column1"]
y =PCA_df["column2"]
z =PCA_df["column3"]
#Plotting the clusters
fig = plt.figure(figsize=(10,8))
ax = plt.subplot(111, projection='3d', label="bla")
ax.scatter(x, y, z, s=40, c=PCA_df["Clusters"], marker='o' , cmap=tab20_5)
ax.set_title("The Plot Of The Clusters")
plt.show()
```



Customer Segmentation- Clustering
Understanding the clusters

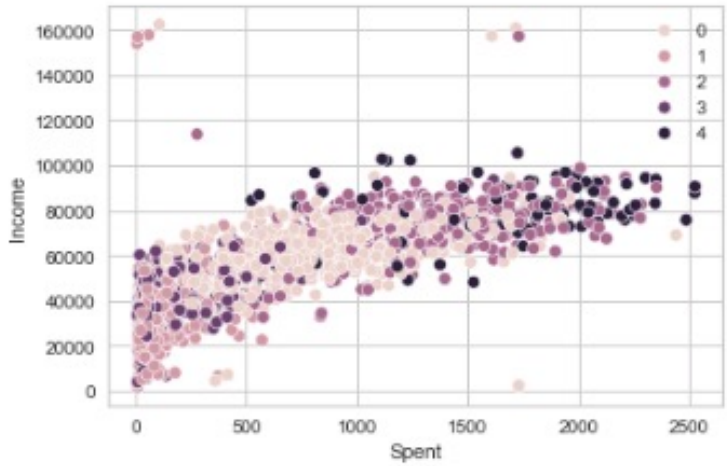
In [26]:

```
# countplot of clusters
#Plotting countplot of clusters
pal = ["#8E7CC3", "#C27BA0","#76A5AF", "#f6b26b", "#741b47"]


pl = sns.countplot(x=customer_data["Clusters"], palette= pal)
pl.set_title("Distribution Of The Clusters")
plt.show()
```



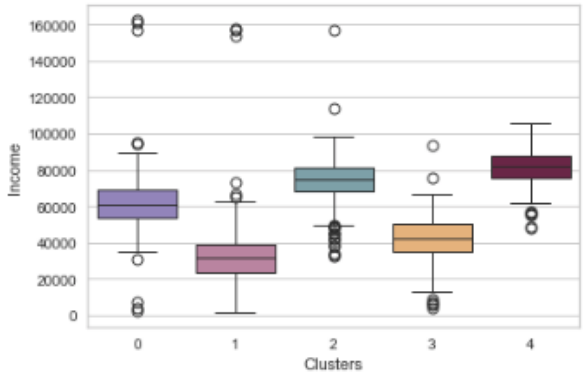
Customer Segmentation- Clustering
Understanding the clusters

In [27]:
```
l = sns.scatterplot(data = customer_data,x=customer_data["Spent"], \
                    y=customer_data["Income"],hue=customer_data["Clusters"])
pl.set_title("Cluster's Profile Based On Income And Spending")
plt.legend()
plt.show()
```



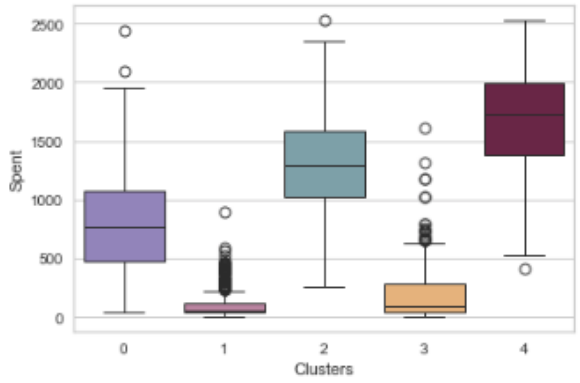
Customer Segmentation- Clustering
Understanding the clusters

In [29]:
```
plt.figure()
pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Income"], palette=pal)
plt.show()
```



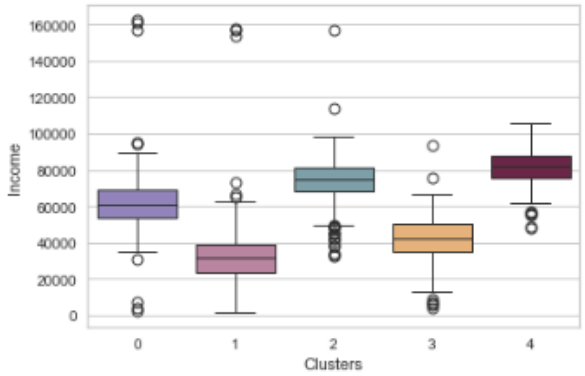
Customer Segmentation- Clustering
Understanding the clusters

In [28]:
```
1 plt.figure()
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Spent"], palette=pal)
3 plt.show()
```



Customer Segmentation- Clustering
Understanding the clusters

In [29]:
```
plt.figure()
pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Income"], palette=pal)
plt.show()
```



Customer Segmentation- Clustering
Understanding the clusters

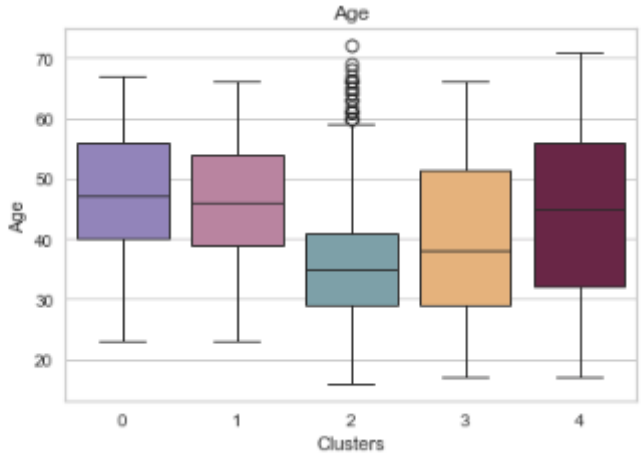
- Observation: clusters patterns in terms of spend and income are similar

Cluster	Median Income	Median Spend	Spend/income
0	$65000	$976	1.50%
1	$45000	$182	0.40%
2	$75000	$1285	1.71%
3	$29000	$57	0.20%
4	$83000	$1772	2.13%

Customer Segmentation- Clustering
Understanding the clusters

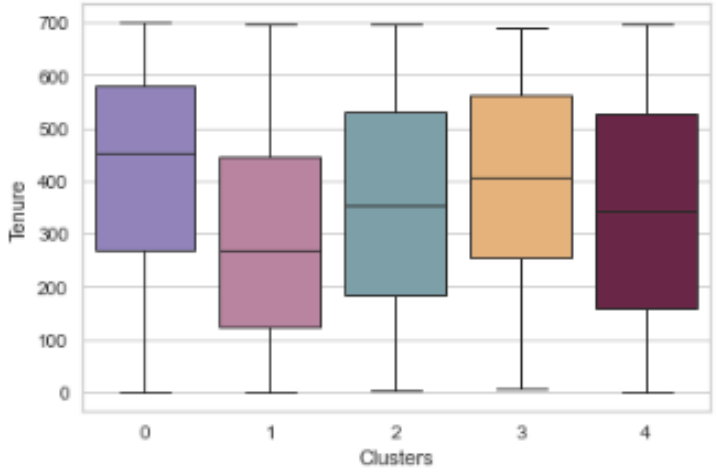
In [30]:

```
1
2 plt.figure()
3 pl=sns.boxplot(y=customer_data["Age"],x=customer_data["Clusters"], palette=pal)
4 pl.set_title("Age")
5 plt.show()
```



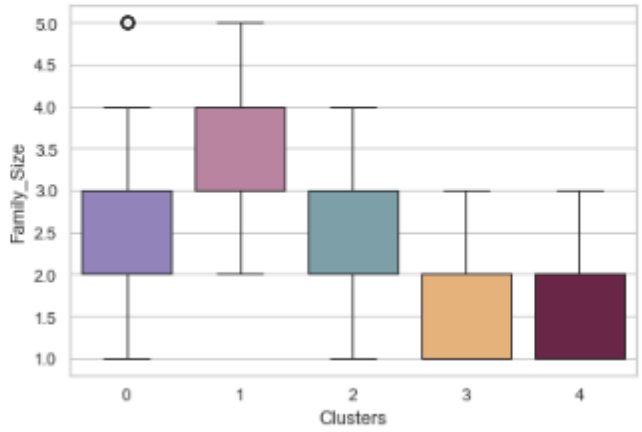
Customer Segmentation- Clustering
Understanding the clusters

In [31]:
```
plt.figure()
pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Tenure"], palette=pal)
plt.show()
```



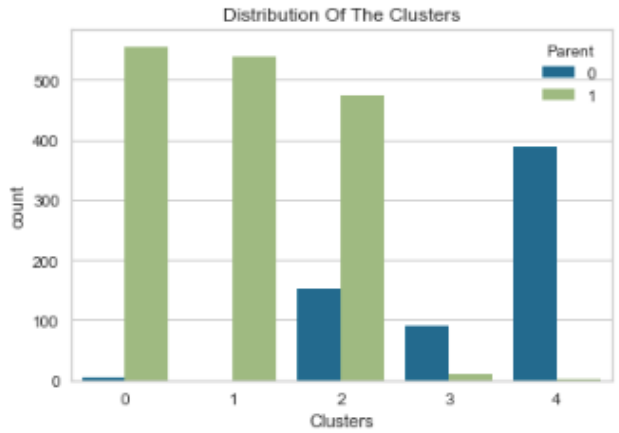
Customer Segmentation- Clustering
Understanding the clusters

In [32]:
```
plt.figure()
pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Family_Size"], palette=pal)
plt.show()
```



Customer Segmentation- Clustering
Understanding the clusters

In [33]:
```
1
2 pl = sns.countplot(x=customer_data["Clusters"], hue=customer_data["Parent"])
3 pl.set_title("Distribution Of The Clusters")
4 plt.show()
```



Customer Segmentation- Clustering
Understanding the clusters

Cluster	Characteristics
0	Highly likely to be parents, High Spender, Large Family, Highest Median Tenure (454 days)
1	Highly likely to be parents, Low Spender, Largest Family Size, Lowest Median Tenure (296 days)
2	Lowest Spender, Large Family
3	Highly unlikely to be parents, Highest Spender, High Spend on wine, Small Family
4	Highly unlikely to be parents, High Spender, Small Family

Patient Segmentation

- In this project, we will conduct unsupervised clustering on patients records from an insurance database.
- Patients' segmentation involves grouping them based on similar behavior/attributes.
- The goal is to categorize patients into segments, optimizing their significance to the insurance company.

Patient Segmentation
Import Libraries

In [1]:

```
#Patients Segmentation Project
import numpy as np
import pandas as pd
from plotly import tools
import matplotlib.pyplot as plt
import seaborn as sns
from string import ascii_letters
from scipy.stats import norm
from scipy.stats import skew
from scipy.stats import pearsonr
from scipy import stats
import statsmodels.api as sm
import plotly.graph_objs as go
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
from statsmodels.sandbox.regression.predstd import wls_prediction_std
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from yellowbrick.cluster import KElbowVisualizer
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings("ignore")
```

Patient Segmentation
Load the dataset

In [2]:
```
# Load the data and do the exploratory data analysis
patient_data = pd.read_csv("patients.csv", sep=",")
patient_data_original=patient_data
print("Number of records:", len(patient_data))
patient_data.head()
```

```
Number of records: 1338
```

Out[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Patient Segmentation

```
In [3]:  1  patient_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1338 non-null   int64  
 1   sex       1338 non-null   object 
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64  
 4   smoker    1338 non-null   object 
 5   region    1338 non-null   object 
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Patient Segmentation

```
In [4]:  1 patient_data.describe().T
```

Out[4]:

	count	mean	std	min	25%	50%	75%	max
age	1338.0	39.207025	14.049960	18.0000	27.00000	39.000	51.000000	64.00000
bmi	1338.0	30.663397	6.098187	15.9600	26.29625	30.400	34.693750	53.13000
children	1338.0	1.094918	1.205493	0.0000	0.00000	1.000	2.000000	5.00000
charges	1338.0	13270.422265	12110.011237	1121.8739	4740.28715	9382.033	16639.912515	63770.42801

Patient Segmentation

In [5]:

```
# Categorical Variable levels
print("Smoker:\n", patient_data["sex"].value_counts(), "\n")
print("Smoker:\n", patient_data["smoker"].value_counts(), "\n")
print("Region:\n", patient_data["region"].value_counts())
```

```
Smoker:
 sex
male      676
female    662
Name: count, dtype: int64

Smoker:
 smoker
no     1064
yes     274
Name: count, dtype: int64

Region:
 region
southeast    364
southwest    325
northwest    325
northeast    324
Name: count, dtype: int64
```

Patient Segmentation

In [6]:
```
# Drop missing data points
patient_data = patient_data.dropna()
print("The total number of recrds after removing the rows with missing values:", len(patient_data))
```

```
The total number of recrds after removing the rows with missing values: 1338
```

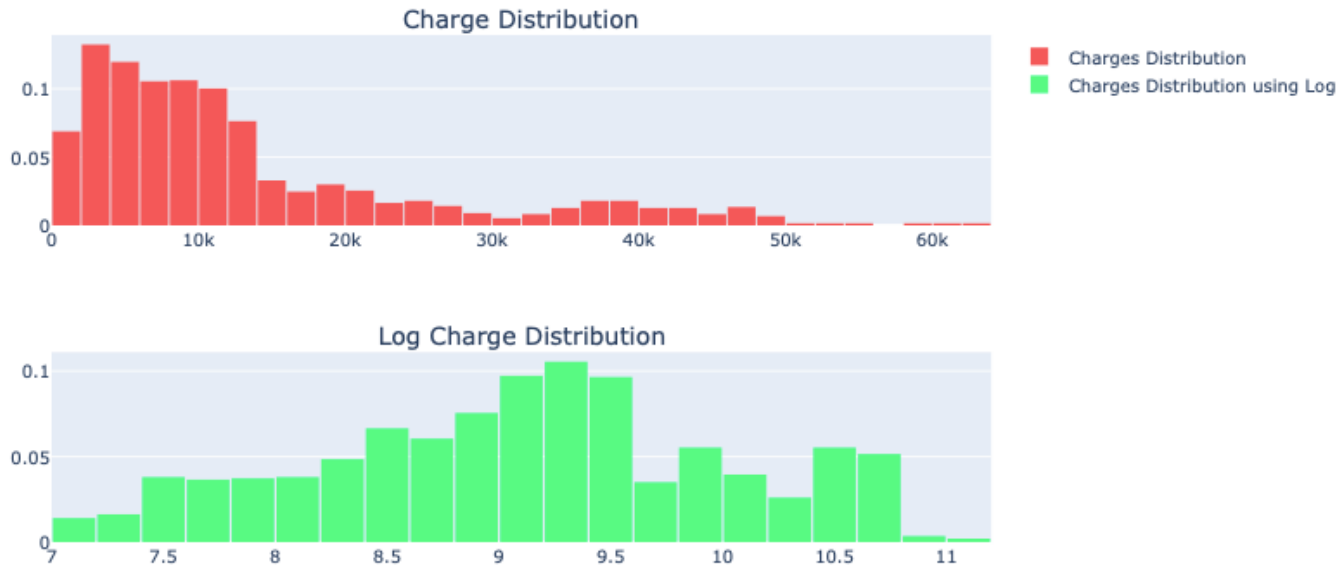
Patient Segmentation

In [7]:

```
#EDA (Exploratory Data Analysis)

# Determine the distribution of charge
charge_dist = patient_data["charges"].values
logcharge = np.log(patient_data["charges"])

trace0 = go.Histogram(
    x=charge_dist,
    histnorm='probability',
    name="Charges Distribution",
    marker = dict(
        color = '#FA5858',
    )
)
trace1 = go.Histogram(
    x=logcharge,
    histnorm='probability',
    name="Charges Distribution using Log",
    marker = dict(
        color = '#58FA82',
    )
)

fig = tools.make_subplots(rows=2, cols=1,
                          subplot_titles=('Charge Distribution','Log Charge Distribution'),
                          print_grid=False)

fig.append_trace(trace0, 1, 1)
fig.append_trace(trace1, 2, 1)

fig['layout'].update(showlegend=True, title='Charge Distribution', bargap=0.05)
iplot(fig, filename='custom-sized-subplot-with-subplot-titles')
```

Patient Segmentation

Charge Distribution

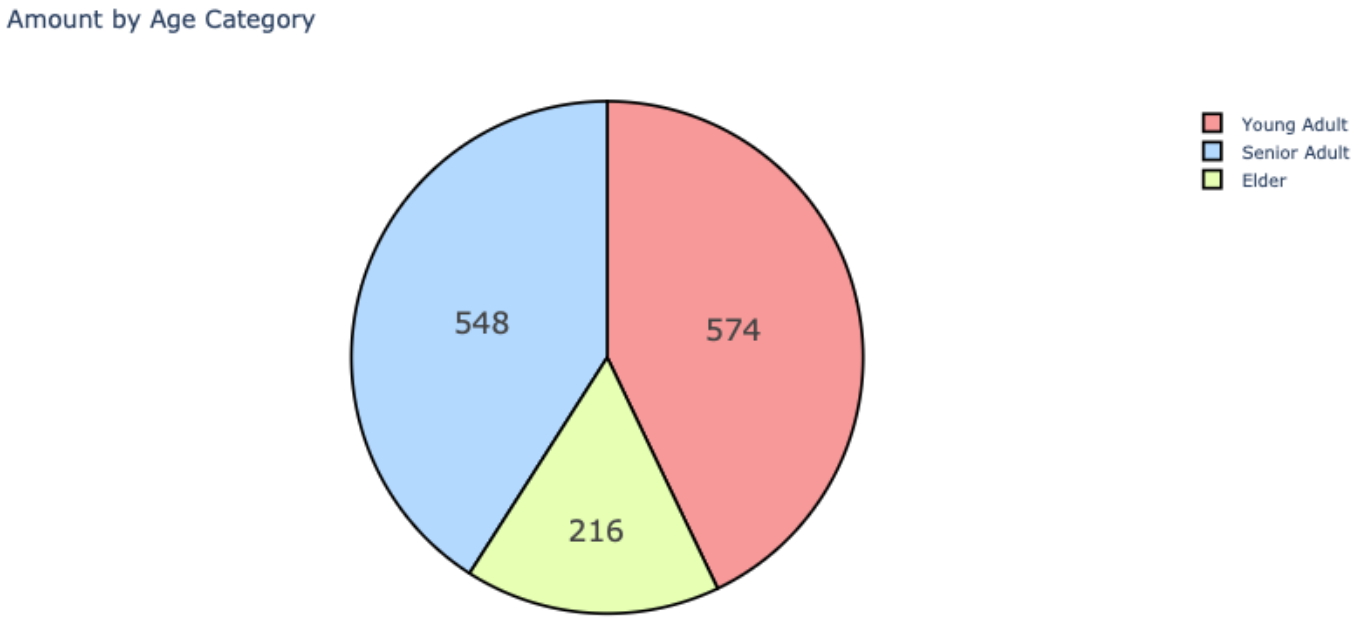


Patient Segmentation

In [8]:

```
patient_data['age_cat'] = np.nan
lst = [patient_data]

for col in lst:
    col.loc[(col['age'] >= 18) & (col['age'] <= 35), 'age_cat'] = 'Young Adult'
    col.loc[(col['age'] > 35) & (col['age'] <= 55), 'age_cat'] = 'Senior Adult'
    col.loc[col['age'] > 55, 'age_cat'] = 'Elder'

labels = patient_data["age_cat"].unique().tolist()
amount = patient_data["age_cat"].value_counts().tolist()

colors = ["#ff9999", "#b3d9ff", " #e6ffb3"]

trace = go.Pie(labels=labels, values=amount,
               hoverinfo='label+percent', textinfo='value',
               textfont=dict(size=20),
               marker=dict(colors=colors,
                           line=dict(color='#000000', width=2)))

data = [trace]
layout = go.Layout(title="Amount by Age Category")
fig = go.Figure(data=data, layout=layout)
iplot(fig, filename='basic_pie_chart')
```

Patient Segmentation

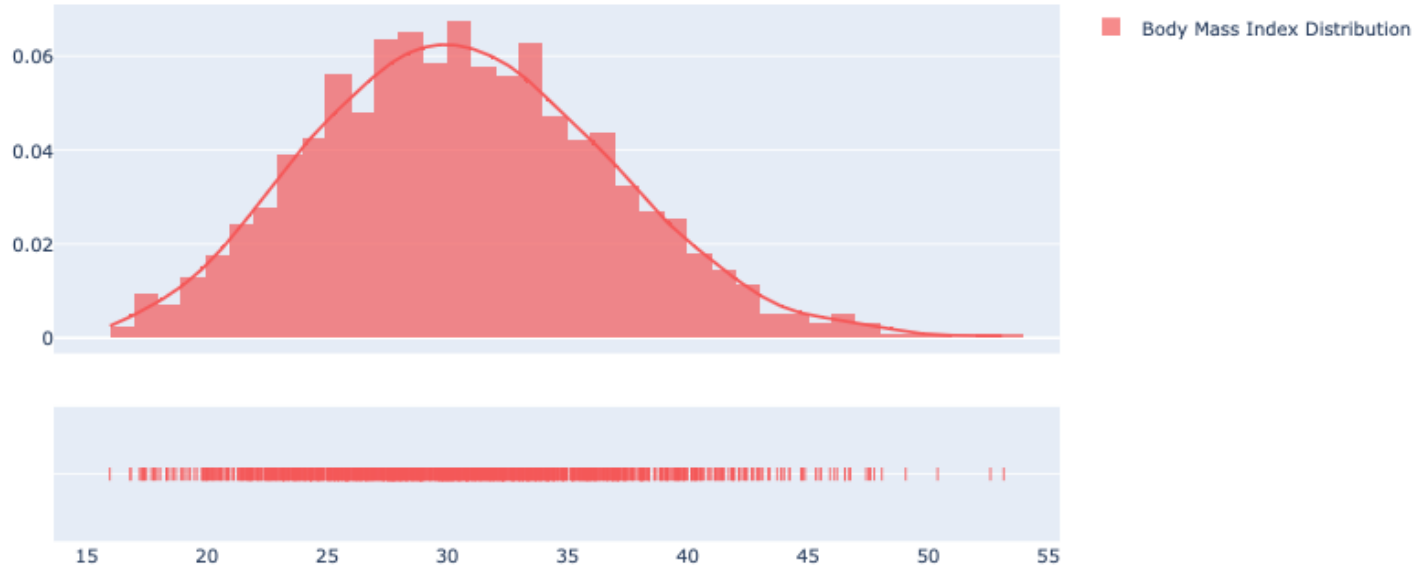


Patient Segmentation

In [9]:

```
# BMI Distribution
import plotly.figure_factory as ff
bmi = [patient_data["bmi"].values.tolist()]
group_labels = ['Body Mass Index Distribution']

colors = ['#FA5858']

fig = ff.create_distplot(bmi, group_labels, colors=colors)
# Add title

iplot(fig, filename='Basic Distplot')
```

Patient Segmentation

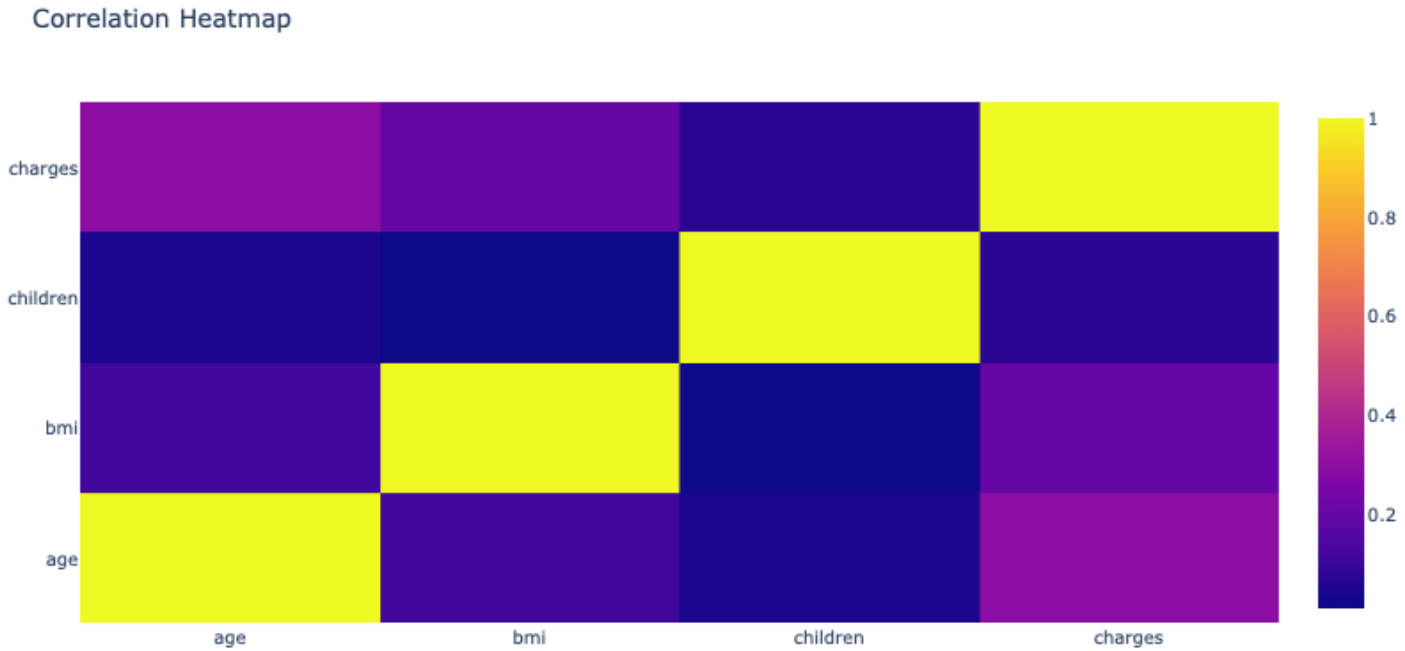


Patient Segmentation

In [10]:

```
# Correlation heatmap of numerical fields
df=patient_data_original.drop(columns=["sex", "smoker","region"])
corr = df.corr()

hm = go.Heatmap(
    z=corr.values,
    x=corr.index.values.tolist(),
    y=corr.index.values.tolist()
)


data = [hm]
layout = go.Layout(title="Correlation Heatmap")

fig = dict(data=data, layout=layout)
iplot(fig, filename='labelled-heatmap')
```

Patient Segmentation



Patient Segmentation

In [11]:
```
# Clustering Pre_Processing Steps
LE=LabelEncoder()
LE=LabelEncoder()
for i in ['age_cat','sex', 'smoker', 'region']:
    patient_data[i]=patient_data[[i]].apply(LE.fit_transform)

```

In [12]:
```
# Scale and save in a new data frame scaled_df
patient_data=patient_data.drop(columns=["charges", "age"])
scaler = StandardScaler()
scaler.fit(patient_data)
scaled_df = pd.DataFrame(scaler.transform(patient_data),columns= patient_data.columns )
scaled_df.head()
```

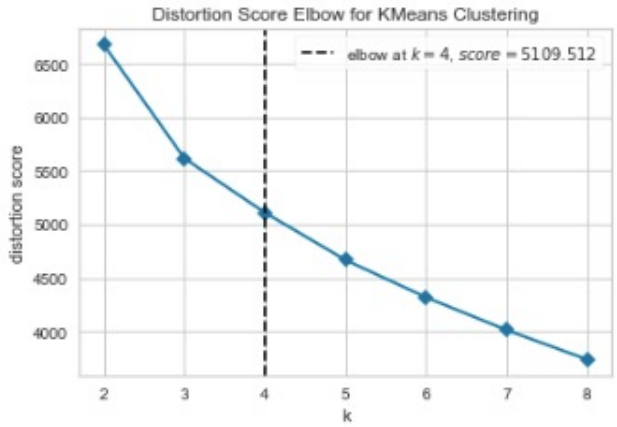
Out[12]:

	sex	bmi	children	smoker	region	age_cat
0	-1.010519	-0.453320	-0.908614	1.970587	1.343905	1.016838
1	0.989591	0.509621	-0.078767	-0.507463	0.438495	1.016838
2	0.989591	0.383307	1.580926	-0.507463	0.438495	1.016838
3	0.989591	-1.305531	-0.908614	-0.507463	-0.466915	1.016838
4	0.989591	-0.292556	-0.908614	-0.507463	-0.466915	1.016838

Patient Segmentation

In [13]:
```
# Elbow method to find numbers of clusters (based on scaled_df)
print('Elbow Method for PCA data set:')
Elbow_M = KElbowVisualizer(KMeans(random_state=42),k=8, timings=False)
Elbow_M.fit(scaled_df)
Elbow_M.show()
```

```
Elbow Method for PCA data set:
```



Out[13]:
```
<Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>
```

Patient Segmentation

In [14]:

```
#Initiating the Agglomerative Clustering model
AC = AgglomerativeClustering(n_clusters=4)
# fit model and predict clusters
yhat_AC = AC.fit_predict(scaled_df)
scaled_df["Clusters"] = yhat_AC
#Adding the Clusters feature to the orignal dataframe.
scaled_df["Clusters"]= yhat_AC
patient_data_original["Clusters"]= yhat_AC
```

Patient Segmentation

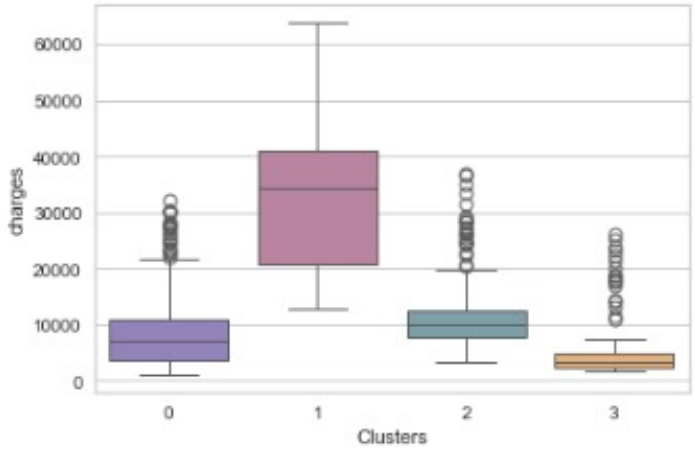
In [15]:

```
# countplot of clusters
#Plotting countplot of clusters
pal = ["#8E7CC3", "#C27BA0","#76A5AF", "#f6b26b", "#741b47"]


pl = sns.countplot(x=patient_data_original["Clusters"], palette= pal)
pl.set_title("Distribution Of The Clusters")
plt.show()
```

Patient Segmentation

In [16]:

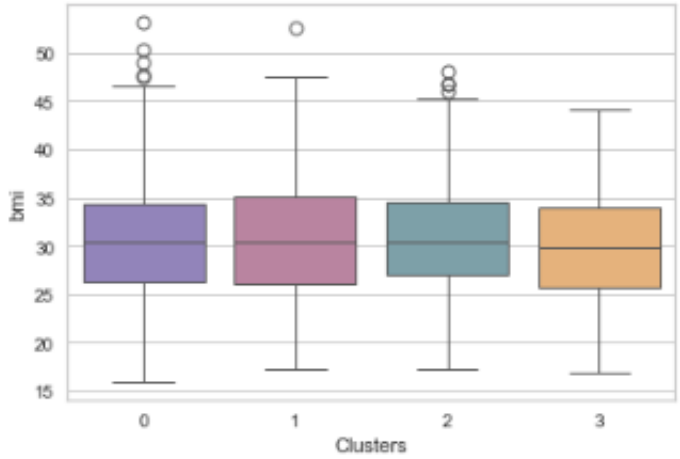
```
1
2 plt.figure()
3 pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["charges"], palette=pal)
4 plt.show()
```



Patient Segmentation

In [17]:

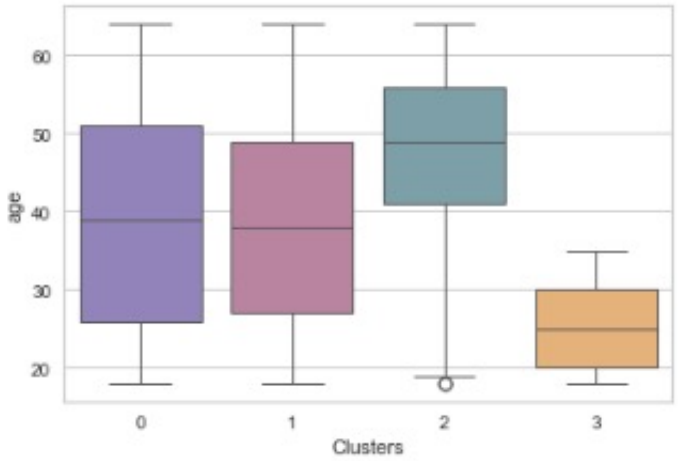
```
1
2 plt.figure()
3 pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["bmi"], palette=pal)
4 plt.show()
```



Patient Segmentation

In [18]:

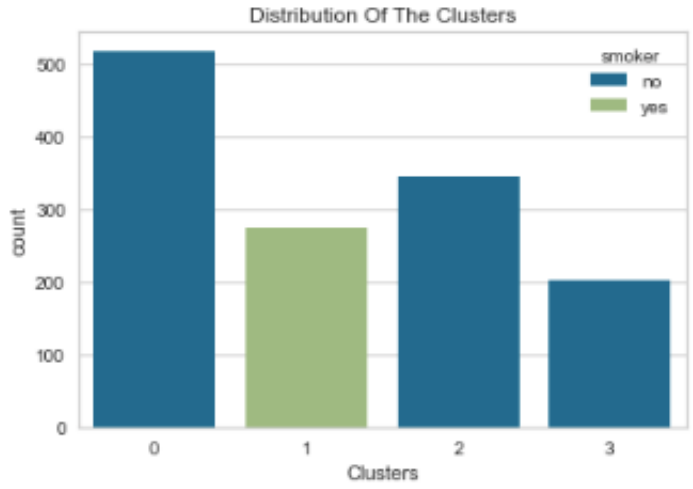
```
1
2 plt.figure()
3 pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["age"], palette=pal)
4 plt.show()
```



Patient Segmentation

In [19]:

```
pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["smoker"])
pl.set_title("Distribution Of The Clusters")
plt.show()
```



Patient Segmentation

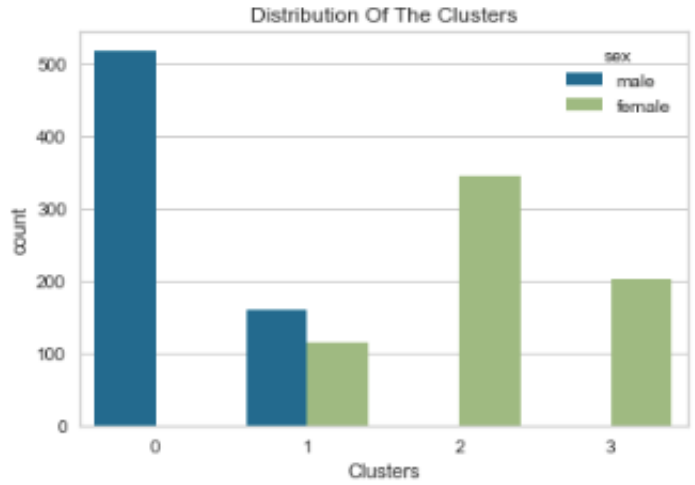
In [20]:

```
1
2 pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["region"])
3 pl.set_title("Distribution Of The Clusters")
4 plt.show()
```

Patient Segmentation

In [21]:

```
pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["sex"])
pl.set_title("Distribution Of The Clusters")
plt.show()
```



Patient Segmentation- Clustering
Understanding the clusters

Cluster	Characteristics
0	median charges around $5,000, median age around 40, non_smokers, male
1	median charges around $35,000, median age around 40, smokers
2	median charges around $10,000, median age around 50, non_smokers, female
3	median charges around $2,500, median age around 25, non_smokers, female

Sample Questions

1. Which clustering method is a centroid based partitioning?
   A. K-Means
   B. DBSCAN
   C. Hierarchical Clustering
   D. A and C

Sample Questions

1. A

Sample Questions

2. Which best describes the primary objective of “topic modeling”?

   A. Identifying clusters of similar documents within a dataset.

   B. Classifying documents into predefined categories.

   C. Extracting key phrases or keywords from textual data.

   D. Uncovering latent themes or topics present in collection of documents.

Sample Questions

2. D

Sample Questions

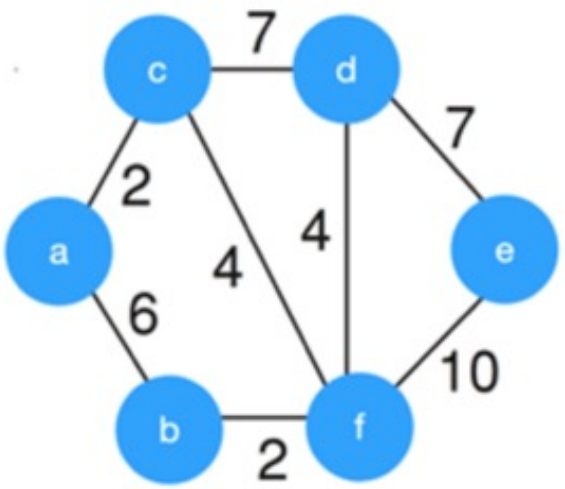
3. What is the weight of the minimum spanning tree using the Kruskal's algorithm for this graph?

A. 24

B. 23

C. 15

D. 19



Sample Questions

3. D

Apply Kruskal algorithm to find MST:

- Start by checking all edges in the graph and selects the smallest edge.
- Then by using these two nodes it constructs a single edge MST.
- Next, it repeats the process and check for the next smallest edge on the graph. It considers to add them into the MST, if the newly selected edge does not create a cycle with previously selected edges.
- This process continues until all nodes are added into the MST.

Sample Questions

4. Consider the following scenario: You are tasked with designing a network to connect a set of cities with the least possible total distance. The distances between the cities are as follows. Given this information, construct the minimum spanning tree for these cities, and determine the total minimum distance required to connect all cities.

	Distance Unit
City A to City B	5
City A to City C	7
City A to City D	9
City B to City C	8
City B to City D	6
City C to City D	5

Sample Questions

4. To construct a minimum spanning tree (MST) for this network of cities and determine the total minimum distance required, we can use algorithms such as Kruskal's or Prim's. I'll demonstrate using Kruskal's algorithm, which is straightforward for this type of problem.

- Here's a step-by-step approach using Kruskal's algorithm:
- **List and Sort the Edges:** First, list all the edges (city connections) and their distances, then sort them in ascending order of distance:
  - A to B: 5
  - C to D: 5
  - B to D: 6
  - A to C: 7
  - B to C: 8
  - A to D: 9
- **Initialize the MST:** Start with an empty minimum spanning tree (MST).

Sample Questions

- **Add Edges:** Add edges to the MST one by one, starting from the shortest, ensuring no cycle is formed:
  - Add A to B (5)
  - Add C to D (5)
  - Add B to D (6)
- At this point, adding any other edge would complete the MST since all cities (A, B, C, D) are connected and form a single connected component. However, for completeness:
  - Adding A to C (7) would not create a cycle with the current tree. But you reach this step only if extending the existing structure fails to include all nodes, which we already have.
- **Total Minimum Distance:** Add up the distances of the edges included in the MST:
- Total Minimum Distance = 5 (A to B) + 5 (C to D) + 6 (B to D) = 16
- Thus, the total minimum distance required to connect all the cities in this network using a minimum spanning tree is 16 distance units.

Sample Questions

5. In one paragraph, explain the Page Rank algorithm and what is the main issue it addresses.

Sample Questions

5. The PageRank algorithm, is a fundamental component of Google's search ranking system. It assigns a numerical value, or "rank," to each element within a hyperlinked set of documents, with the intent of measuring its relative importance within the set. The core concept is that a page is considered important if it is linked to by other important pages. PageRank iterates this idea by treating each incoming link to a page as a vote of support and weighting these votes by the rank of the source page, thereby creating a recursive metric of importance. The main issue PageRank addresses is the problem of ranking search results based on their relevance and importance, beyond simple keyword matching, thereby improving the accuracy of search engine results by leveraging the collective knowledge embedded in the link structure of the web.

Sample Questions

6. A dataset representing the monthly sales performance of a retail store, with features including the number of items sold, total revenue, advertising expenses, and customer satisfaction scores. Applying PCA to this dataset, which of the following statements best describes the role of principal components?

A. Principal components represent the individual features such as items sold, revenue, advertising expenses, and customer satisfaction in their original form.
B. Principal components are linear combinations of the original features, capturing the maximum variance in the data and providing new, uncorrelated variables.
C. Principal components quantify the absolute values of each feature and help identify the most significant factors influencing sales performance.
D. Principal components indicate the presence of outliers in the dataset and help in their identification and removal.

Sample Questions

6. B

References

- https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis